

Supplementary material for: PointProgrammerNet: A Fixed-state additive FWP network for streaming point-cloud recognition

Dian Yu^a

^a *University of New South Wales, Sydney, NSW 2052, Australia*

The main paper specifies the point encoder, spatial addresses, additive outer-product state, task readouts, and principal experiments. This supplement records only implementation details needed for exact reproduction or for checking the stated properties.

1. Exact readout and numerical stabilization

1.1. Segmentation

For branch s , a query coordinate $\mathbf{q}$ reads the fixed state as

$$\mathbf{a}_s(\mathbf{q}) = \mathbf{k}_s(\mathbf{q})^\top \mathbf{S}_s \in \mathbb{R}^{170}.$$

The first component is the read mass $\mu_s(\mathbf{q})$. The implementation prevents unstable division in nearly empty regions with

$$\epsilon_s = 10^{-3} \text{mean}_{\mathbf{q}'} \mu_s(\mathbf{q}') + 10^{-20}, \quad \tilde{\mu}_s(\mathbf{q}) = \max(\mu_s(\mathbf{q}), \epsilon_s). \quad (1)$$

The mean is taken over the query coordinates of one sample and is detached from the gradient. The maximum in Equation (1) is only a scalar numerical floor. It neither compares point features nor performs global maximum pooling.

The remaining columns of $\mathbf{a}_s(\mathbf{q})$ contain weighted coordinate, feature, cross-product, coordinate-square, and feature-square sums. Dividing them by $\tilde{\mu}_s$ gives $\bar{\mathbf{x}}_s, \bar{\mathbf{g}}_s, E_{xg,s}, E_{xx,s}, E_{gg,s}$. The decoded descriptor is

$$\mathbf{d}_s(\mathbf{q}) = [\bar{\mathbf{g}}_s, \bar{\mathbf{x}}_s - \mathbf{q}, \log \tilde{\mu}_s, \text{vec}(E_{xg,s} - \mathbf{q}\bar{\mathbf{g}}_s^\top), \text{vech}(E_{xx,s} - \bar{\mathbf{x}}_s\bar{\mathbf{x}}_s^\top), \sqrt{[E_{gg,s} - \bar{\mathbf{g}}_s \odot \bar{\mathbf{g}}_s]_+ + 10^{-8}}]. \quad (2)$$

Its block dimensions are 32, 3, 1, 96, 6, 32, totalling 170. The covariance and standard-deviation terms retain variation lost by a mean. The clamp $[\cdot]_+$ removes small negative variances caused by floating-point roundoff.

Each branch independently applies the same $170 \rightarrow 160 \rightarrow 160$ BatchNorm-ReLU decoder. The three outputs are concatenated with a 32-dimensional coordinate feature and the 16-dimensional ShapeNetPart category code. The segmentation MLP is $528 \rightarrow 256 \rightarrow 128 \rightarrow 50$, with BatchNorm, ReLU, and dropout 0.3 after its first two affine layers. Queries are processed independently; there is no pooling or interaction over decoded query points.

1.2. Classification

Classification retains no coordinates or point features. For row j , the 170 stored entries are partitioned as

$$[\mu_{sj}, \mathbf{p}_{sj}, \mathbf{f}_{sj}, \text{vec}(\mathbf{C}_{sj}), \text{vech}(\mathbf{Q}_{sj}), \mathbf{u}_{sj}],$$

where the blocks are mass, coordinate sum, feature sum, coordinate-feature sum, symmetric coordinate second moment, and squared-feature sum. Row normalization uses

$$\tilde{\mu}_{sj} = \max\left(\mu_{sj}, 10^{-4}R^{-1}\sum_{u=1}^R|\mu_{su}| + 10^{-20}\right). \quad (3)$$

The lower bound is detached during back-propagation. With $\bar{\mu}_s = R^{-1} \sum_u \tilde{\mu}_{su}$, division by $\tilde{\mu}_{sj}$ produces

$$\begin{aligned} \mathbf{d}_{sj}^{\text{row}} = & [\bar{\mathbf{g}}_{sj}, \bar{\mathbf{x}}_{sj}, \log(\tilde{\mu}_{sj}/\bar{\mu}_s), \text{vech}(\text{Cov}_{xx,sj}), \\ & \text{Std}_{g,sj}, \text{vec}(\text{Cov}_{xg,sj})] \in \mathbb{R}^{170}. \end{aligned} \quad (4)$$

The six block dimensions are 32, 3, 1, 6, 32, 96.

Each branch has an independent $170 \rightarrow 128 \rightarrow 128$ LayerNorm–GELU row encoder. Two learned score columns form two softmax-weighted summaries over the fixed 128 rows. Their concatenation passes through an affine–LayerNorm–GELU map to give $\mathbf{h}_s \in \mathbb{R}^{256}$. This softmax always processes 128 state rows and therefore does not grow with the number of observed points.

The middle summary is the reference. Fine and broad summaries produce

$$\boldsymbol{\delta} = \sigma(a) \tanh(W[\mathbf{h}_f; \mathbf{h}_b] + \mathbf{b}), \quad \mathbf{z}_{\text{cls}} = h_{\text{cls}}(\text{LN}(\mathbf{h}_m + \boldsymbol{\delta})).$$

The residual projection is initialized to zero and $\sigma(a)$ to 0.1. The classifier h_{cls} is $256 \rightarrow 256 \rightarrow 40$, with GELU and dropout 0.3. All radii are learned. Only the middle radius uses a $25\times$ learning-rate multiplier and zero weight decay; all other parameters use the base optimizer settings.

2. Proof details

Let $F_s(\mathbf{x}) = \mathbf{k}_s(\mathbf{x})\mathbf{v}_s(\mathbf{x})^\top$ be one point’s rank-one FWP write. The encoder state is $\mathbf{S}_s(X) = \sum_{\mathbf{x} \in X} F_s(\mathbf{x})$.

Proposition 1 (Order, chunks, and devices). *For point multisets A, B ,*

$$\mathbf{S}_s(A \uplus B) = \mathbf{S}_s(A) + \mathbf{S}_s(B),$$

and the state is invariant to point order. Devices using identical trained encoders, coordinate normalization, and reference frames therefore obtain the centralized state by adding their independently encoded states.

Proof. Both sides expand into the same finite collection of per-point matrices $F_s(\mathbf{x})$. Reordering or regrouping a finite sum does not change it in real arithmetic. Float32 results may differ only through accumulation order. $\square$

The same identity permits subtraction when the state of the removable subset is available. Exponential forgetting follows from

$$\mathbf{S}_{s,t} = \rho \mathbf{S}_{s,t-1} + \mathbf{S}_s(X_t), \quad 0 \leq \rho \leq 1.$$

Subtraction removes an aggregate contribution; it does not reconstruct its individual points.

Proposition 2 (Retained memory and update cost). *With R_s rows and D_s columns per branch, retained memory is $O(\sum_s R_s D_s)$, independent of history length. Encoding M new points costs $O(M \sum_s R_s D_s)$ and does not revisit earlier observations.*

Proof. Only the fixed matrices persist. A new chunk contributes new outer products of the same dimensions regardless of the number of earlier chunks. $\square$

For evidence strength $t \in [0, 1]$,

$$\mathbf{S}_s(t) = \mathbf{S}_s^{\text{old}} + t F_s(\mathbf{x})$$

is affine in t . Gaussian addresses, softmax, positive-floor normalization, affine layers, ReLU or GELU, and stabilized square roots are continuous. Hence

classification logits and fixed-query segmentation logits vary continuously and piecewise smoothly with t . For a fixed state, segmentation logits have the same property with respect to the query coordinate. A hard argmax label can still change discontinuously at a decision boundary.

3. Controlled screens and reproducibility

All screens use the same data split and optimization protocol within a comparison, five epochs, and seeds 0, 1, and 2. Table 1 reports paired changes of the retained design relative to the named alternative. These short runs select structural choices; the main tables report final 20-epoch results.

Table 1: Controlled five-epoch screens. Positive changes favor the retained design.

Retained choice	Paired alternative	Paired change
Seg.: equal decoders	unequal middle decoder	+0.30 $\pm$ 0.30 instance; +0.15 $\pm$ 0.29 category
Seg.: distinct radius starts	equal radius starts	+0.74 $\pm$ 0.42 instance; −0.05 $\pm$ 0.33 category
Seg.: common normalized key	mixed key definitions	+0.35 $\pm$ 0.16 instance; +0.85 $\pm$ 0.37 category
Cls.: middle residual	direct concatenation	+2.67 $\pm$ 1.60 clean; +3.78 $\pm$ 1.71 shifted
Cls.: 25 $\times$ middle-radius LR	base radius LR	+0.13 $\pm$ 1.03 clean; +0.91 $\pm$ 0.33 shifted

The segmentation columns report instance and category mIoU. Equal decoders also remove 24,480 parameters. The classification learning-rate choice

is retained for its consistent shifted-accuracy gain rather than its variable clean-accuracy change.

The released minimal code contains the 341,909-parameter segmenter and 529,426-parameter classifier, dataset loaders, training loops, state-operation checks, and exact three-seed reference values. `CODE_REPRODUCTION.md` specifies the environment, dataset layout, commands, and streaming API. Additivity tests should run in evaluation mode because BatchNorm then uses the fixed running statistics assumed by the proof.